

ATIVIDADE 11

Análise e Evolução de Aplicação Flutter

Group 6

Seção 01 — Latência: Diagnóstico e Evidências

1. Objetivo

Esta seção tem como objetivo identificar os pontos de lentidão presentes no projeto flutter_cache, registrar evidências técnicas dos problemas e propor hipóteses sobre suas causas. O app foi propositalmente construído com falhas de latência, ausência de cache e responsividade bloqueada.

2. Ambiente de Teste

Campo	Valor
Dispositivo / Emulador	Emulador Android — Pixel 7 Pro (AVD Manager)
Sistema Operacional	Android 14 (API Level 34)
Versão do Flutter	Flutter 3.22.x (stable channel)
Versão do Dart	Dart 3.4.x
Modo de execução	Debug Mode (flutter run)
Conexão de rede simulada	Wi-Fi padrão (sem throttling adicional)
Repositório analisado	github.com/jeffersonspeck/flutter_cache

3. Problemas de Latência Identificados

A análise estática do código-fonte (lib/main.dart) revelou três focos principais de latência, todos comentados explicitamente pelo autor do projeto como problemas intencionais.

3.1 — Atraso Artificial de 2 Segundos no Carregamento Inicial

Localização no código: método `loadProducts()` — linha com `await Future.delayed(const Duration(seconds: 2))`

Descrição do problema:

Antes de realizar a requisição HTTP à API `dummyjson.com/products`, o método `loadProducts()` executa um delay fixo de 2 segundos. Esse atraso é inserido programaticamente sem qualquer finalidade funcional — seu único efeito é bloquear a interface do usuário durante esse período.

Passos para reprodução:

1. Inicializar o aplicativo no emulador ou dispositivo.
2. Observar a tela inicial — um `CircularProgressIndicator` será exibido imediatamente.
3. Aguardar: a lista de produtos não aparece por pelo menos 2 segundos + tempo de rede.
4. Medir o tempo total de espera: tipicamente entre 3 e 5 segundos até a lista ser exibida.

Evidências:

- Tela/fluxo afetado: `ProductListPage` — tela inicial do aplicativo
- Tempo percebido (aproximado): 3 a 5 segundos do lançamento até a exibição da lista
- Causa confirmada no código: `await Future.delayed(const Duration(seconds: 2))` na linha anterior à chamada `http.get()`

3.2 — Recarregamento Completo da Lista ao Retornar dos Detalhes

Localização no código: método `openDetails()` — chamada `await loadProducts()` após `Navigator.push()`

Descrição do problema:

Ao tocar em qualquer produto da lista para visualizar seus detalhes, o usuário é navegado para `ProductDetailPage`. Ao pressionar o botão Voltar (back), o método `openDetails()` executa novamente `loadProducts()` — repetindo o delay artificial de 2 segundos e fazendo nova requisição HTTP completa à API remota. Isso descarta os dados já carregados sem necessidade.

Passos para reprodução:

5. Com a lista de produtos já carregada, tocar em qualquer item.
6. Visualizar a tela de detalhes do produto.
7. Pressionar o botão Voltar.
8. Observar: a tela retorna ao loading bloqueante (`CircularProgressIndicator`) e todos os produtos desaparecem por 3 a 5 segundos enquanto a lista é recarregada do zero.

Evidências:

- Tela/fluxo afetado: Transição `ProductDetailPage` → `ProductListPage`
- Tempo percebido (aproximado): 3 a 5 segundos adicionais a cada retorno da tela de detalhes
- Causa confirmada no código: `await loadProducts()` chamado incondicionalmente no `openDetails()`, após o pop da rota

3.3 — Imagens Carregadas Sem Cache e Sem Placeholder

Localização no código: `Image.network()` na `ProductListPage` (thumbnail) e `ProductDetailPage` (PageView de imagens)

Descrição do problema:

Todas as imagens dos produtos são carregadas via `Image.network()` sem nenhuma estratégia de cache explícita, sem placeholder durante o carregamento e sem controle de prioridade. Cada vez que o widget é renderizado — inclusive nos recarregamentos forçados descritos nos itens 3.1 e 3.2 — as imagens precisam ser baixadas novamente da rede. Na tela de detalhes, o `PageView` pode conter múltiplas imagens pesadas carregadas simultaneamente.

Passos para reprodução:

9. Após a lista carregar, rolar rapidamente pela lista de produtos.
10. Observar o espaço vazio (sem placeholder) nos thumbnails enquanto as imagens são baixadas.
11. Voltar da tela de detalhes para acionar o recarregamento — observar as imagens sumindo e reaparecendo.

Evidências:

- Tela/fluxo afetado: `ProductListPage` (thumbnails) e `ProductDetailPage` (galeria de imagens)
- Comportamento observado: Flash branco ou ícone de erro temporário nos thumbnails a cada rebuild
- Causa confirmada no código: `Image.network()` sem `cacheWidth`, sem `CachedNetworkImage` e sem `loadingBuilder`

4. Resumo dos Problemas e Impacto

Problema	Tempo de Impacto	Frequência
Delay artificial de 2s	2s fixos + latência de rede (~1-3s)	Toda abertura do app e todo refresh

Problema	Tempo de Impacto	Frequência
Reload ao voltar dos detalhes	3 a 5s adicionais	Toda navegação para detalhes e retorno
Imagens sem cache	Variável (depende da rede)	Toda renderização da lista ou rebuild

5. Hipóteses Técnicas

Com base na análise estática do código, as hipóteses abaixo explicam as causas raiz dos problemas de latência:

Problema 3.1 — Delay artificial

- O `await Future.delayed()` foi inserido intencionalmente para simular uma situação onde o desenvolvedor deixa um timer de teste no código de produção — prática comum em projetos onde não há separação entre código de desenvolvimento e produção.
- Ausência de cache local (`SharedPreferences`, `Hive`, `SQLite` ou similares) força a requisição HTTP a sempre ser feita do zero, sem possibilidade de resposta imediata com dados previamente carregados.

Problema 3.2 — Reload ao retornar

- O método `openDetails()` chama `loadProducts()` incondicionalmente após o pop da navegação. Isso indica ausência de estado compartilhado ou gerenciamento de estado (como `Provider`, `Riverpod`, `Bloc` ou mesmo um `InheritedWidget`) que manteria os dados em memória entre as telas.
- Sem uma camada de repositório ou serviço separado do widget, o estado da lista fica acoplado ao ciclo de vida do `State<ProductListPage>`, sendo descartado a cada reconstrução forçada.

Problema 3.3 — Imagens sem cache

- O Flutter possui um cache interno de imagens baseado em memória (`ImageCache`), mas ele não persiste entre sessões e pode ser descartado sob pressão de memória. Nenhum mecanismo de cache persistente em disco (como o pacote `cached_network_image`) foi implementado.
- A ausência de `loadingBuilder` e `cacheWidth` no `Image.network()` causa saltos visuais e impede o Flutter de redimensionar as imagens antes de decodificá-las, gerando trabalho desnecessário na thread de UI.

6. Impacto para o Usuário

Os três problemas combinados causam uma experiência de uso significativamente degradada:

- Na abertura do app: o usuário enfrenta mais de 3 segundos de tela de loading bloqueante, sem nenhuma indicação de progresso parcial. Estudos de UX indicam que atrasos acima de 3 segundos aumentam expressivamente a taxa de abandono.
- Durante a navegação: ao voltar da tela de detalhes, o usuário perde o contexto visual — a lista some e precisa ser recarregada integralmente, fazendo com que o app pareça instável ou travado.
- Rolagem da lista: thumbnails com espaços vazios (sem skeleton ou placeholder) criam uma percepção de app incompleto ou com má conexão, mesmo quando a rede está disponível.
- Consumo desnecessário de dados: cada retorno da tela de detalhes gera uma nova requisição HTTP completa de 30 produtos, desperdiçando banda e bateria do dispositivo.

Efeito consolidado: o usuário percebe o app como lento, pouco responsivo e inconsistente — mesmo que a API remota responda rapidamente. A causa não é a infraestrutura, mas decisões arquiteturais no código da aplicação.

7. Sugestões Iniciais de Melhoria

As sugestões abaixo são propostas de solução para os problemas identificados. A implementação detalhada será abordada nas demais seções da atividade.

Para o Problema 3.1 — Delay artificial

- Remover imediatamente o `await Future.delayed()` do código de produção.
- Implementar cache local com `SharedPreferences` ou `Hive` para exibir dados previamente carregados enquanto a atualização ocorre em segundo plano (estratégia `stale-while-revalidate`).

Para o Problema 3.2 — Reload ao retornar

- Introduzir uma camada de serviço/repositório (ex.: `ProductRepository`) que mantenha os dados em memória durante a sessão.
- Adotar gerenciamento de estado com `Provider` ou `Riverpod` para que a lista não dependa do ciclo de vida do widget.
- Remover a chamada `loadProducts()` do `openDetails()` — o reload só deve ocorrer quando explicitamente solicitado pelo usuário (botão de refresh).

Para o Problema 3.3 — Imagens sem cache

- Adicionar o pacote `cached_network_image` para cache persistente em disco das imagens.

- Implementar placeholder e errorWidget adequados para eliminar flashes visuais.
- Utilizar cacheWidth nos thumbnails para reduzir consumo de memória na decodificação.